

■ LEVEL 19 • 1D DP

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	One variable describes the state. Each state depends on earlier states; often space can be optimized.
Reference Examples	Climbing Stairs • House Robber • Min Cost Climbing Stairs • Decode Ways • Maximum Subarray
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 19 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Min Cost Climbing Stairs	1D DP	#746
2	House Robber II	Circular 1D DP	#213
3	Decode Ways	1D DP	#91
4	Word Break	1D DP + Set	#139
5	Maximum Product Subarray	DP State	#152
6	Maximum Subarray	Kadane 1D DP	#53
7	Climbing Stairs	1D DP	70
8	House Robber	1D DP	198
9	Min Cost Climbing Stairs	1D DP	746
10	Decode Ways	1D DP	91

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Min Cost Climbing Stairs

Pattern: 1D DP

LeetCode: #746

■ Input	cost array
■ Output	minimum cost to reach the top
■ Constraints	$2 \leq n \leq 1000$
■ Main Idea / Pattern	To reach step i , come from $i-1$ or $i-2$ and pay the chosen step cost.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int minCostClimbingStairs(vector<int>& cost) {  
    int a = 0, b = 0;  
    for (int i = 2; i <= cost.size(); i++) {  
        int c = min(b + cost[i - 1],  
                    a + cost[i - 2]);  
        a = b;  
        b = c;  
    }  
    return b;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 House Robber II

Pattern: Circular 1D DP

LeetCode: #213

■ Input	circular house amounts
■ Output	maximum money without robbing adjacent houses
■ Constraints	$1 \leq n \leq 100$
■ Main Idea / Pattern	Cannot rob both first and last. Solve two linear ranges and take the larger.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int solve(vector<int>& a, int l, int r) {
    int prev2 = 0, prev1 = 0;
    for (int i = l; i <= r; i++) {
        int cur = max(prev1, prev2 + a[i]);
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}

int rob(vector<int>& nums) {
    if (nums.size() == 1) return nums[0];
    return max(solve(nums, 0, nums.size() - 2),
               solve(nums, 1, nums.size() - 1));
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Decode Ways

Pattern: 1D DP

LeetCode: #91

■ Input	digit string
■ Output	number of decodings
■ Constraints	valid digit string
■ Main Idea / Pattern	A position can use one digit if nonzero, or two digits if the pair is 10..26.
■■ Time Complexity	O(n)
■ Space Complexity	O(1)

■ Optimal C++ Code

```
int numDecodings(string s) {
    int prev2 = 1, prev1 = 1;
    for (int i = 1; i <= s.size(); i++) {
        int cur = 0;
        if (s[i - 1] != '0')
            cur += prev1;
        if (i >= 2) {
            int x = stoi(s.substr(i - 2, 2));
            if (x >= 10 && x <= 26)
                cur += prev2;
        }
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Word Break

Pattern: 1D DP + Set

LeetCode: #139

■ Input	string and dictionary
■ Output	whether string can be segmented into dictionary words
■ Constraints	word lengths bounded
■ Main Idea / Pattern	dp[i] means prefix ending at i can be formed. Try every earlier split.
■ Time Complexity	$O(n^2 \cdot L)$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
bool wordBreak(string s, vector<string>& wordDict) {
    unordered_set<string> dict(wordDict.begin(), wordDict.end());
    int n = s.size();
    vector<bool> dp(n + 1);
    dp[0] = true;

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && dict.count(s.substr(j, i - j))) {
                dp[i] = true;
                break;
            }
        }
    }
    return dp[n];
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Maximum Product Subarray

Pattern: DP State

LeetCode: #152

■ Input	integer array
■ Output	maximum product of a contiguous subarray
■ Constraints	$1 \leq n \leq 2 * 10^4$
■ Main Idea / Pattern	Track both maximum and minimum product ending here because a negative value can swap them.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int maxProduct(vector<int>& nums) {  
    int mx = nums[0], mn = nums[0], ans = nums[0];  
    for (int i = 1; i < nums.size(); i++) {  
        if (nums[i] < 0)  
            swap(mx, mn);  
  
        mx = max(nums[i], mx * nums[i]);  
        mn = min(nums[i], mn * nums[i]);  
  
        ans = max(ans, mx);  
    }  
    return ans;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Maximum Subarray

Pattern: Kadane 1D DP

LeetCode: #53

■ Input	integer array
■ Output	maximum subarray sum
■ Constraints	non-empty array
■ Main Idea / Pattern	The state is the best sum ending at i: $\max(\text{nums}[i], \text{previous} + \text{nums}[i])$.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int maxSubArray(vector<int>& nums) {  
    int cur = nums[0], ans = nums[0];  
    for (int i = 1; i < nums.size(); i++) {  
        cur = max(nums[i], cur + nums[i]);  
        ans = max(ans, cur);  
    }  
    return ans;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Climbing Stairs

Pattern: 1D DP

LeetCode: 70

Input	n
Output	number of ways to reach the top
Constraints	1 <= n <= 45
Main Idea / Pattern	The last step is either from n-1 or n-2.
Time Complexity	O(n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    int climbStairs(int n) {
        int a = 1;
        int b = 1;
        for (int i = 2; i <= n; i++) {
            int c = a + b;
            a = b;
            b = c;
        }
        return b;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 House Robber

Pattern: 1D DP

LeetCode: 198

Input	nums
Output	maximum money without robbing adjacent houses
Constraints	1 <= nums.size() <= 100
Main Idea / Pattern	At each house, choose skip or rob it plus the best answer two positions back.
Time Complexity	O(n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    int rob(vector<int>& nums) {
        int prev2 = 0;
        int prev1 = 0;
        for (int money : nums) {
            int cur = max(prev1, prev2 + money);
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Min Cost Climbing Stairs

Pattern: 1D DP

LeetCode: 746

Input	cost
Output	minimum cost to reach the top
Constraints	2 <= cost.size() <= 1000
Main Idea / Pattern	Each position can be reached from one of the previous two positions.
Time Complexity	O(n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    int minCostClimbingStairs(vector<int>& cost) {
        int a = 0;
        int b = 0;
        for (int i = 2; i <= cost.size(); i++) {
            int c = min(b + cost[i - 1], a + cost[i - 2]);
            a = b;
            b = c;
        }
        return b;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Decode Ways

Pattern: 1D DP

LeetCode: 91

Input	s
Output	number of decodings
Constraints	1 <= s.size() <= 100
Main Idea / Pattern	At each position, take one valid digit or one valid two-digit number.
Time Complexity	O(n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    int numDecodings(string s) {
        int next2 = 1;
        int next1 = s.back() != '0';
        for (int i = s.size() - 2; i >= 0; i--) {
            int cur = 0;
            if (s[i] != '0') {
                cur = next1;
                int two = stoi(s.substr(i, 2));
                if (two >= 10 && two <= 26) {
                    cur += next2;
                }
            }
            next2 = next1;
            next1 = cur;
        }
        return next1;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.